



Presented by:

Group2

Swostika Lama,

Puntawat Subhamani,

Jiya Jameela Kanhirathan

Poyil,

Sachin Bhandari

PhishDetectAI

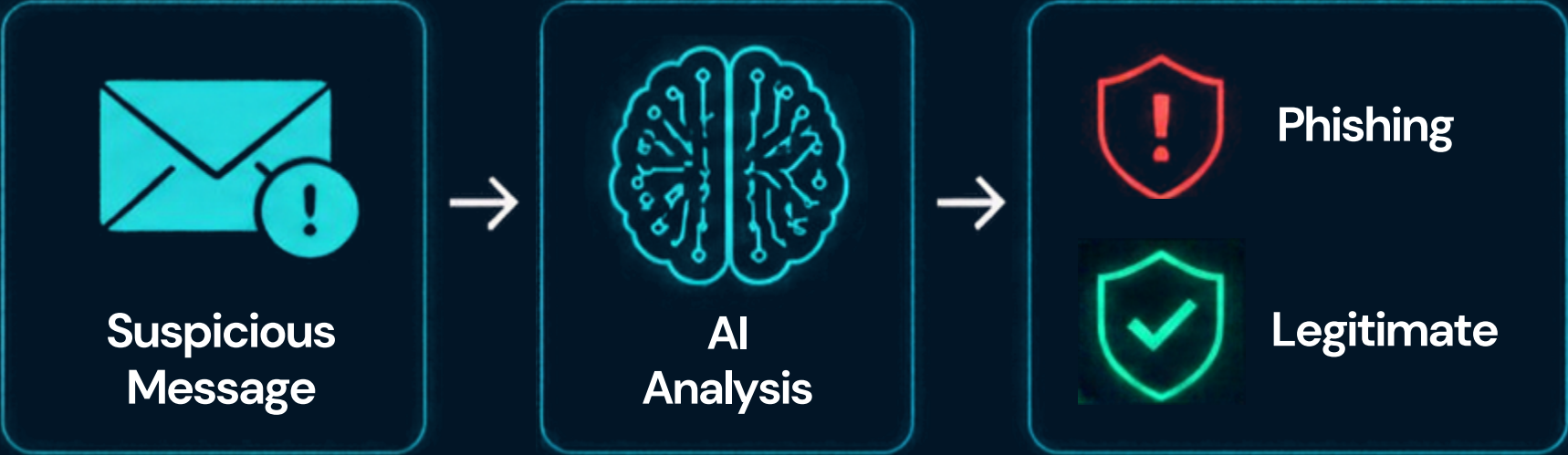
A Phishing Detection System

Domain: Cybersecurity / Email & Message Security

Project Overview

A Phishing Detection System

	Domain:	Cybersecurity & Natural Language Processing (NLP)
	Problem to solve:	Rising phishing attacks use fraudulent emails and messages to trick users into revealing sensitive data or clicking malicious links; traditional filters can miss advanced attacks.
	Intended users / use case:	Everyday individuals, students, employees, and small organizations needing a simple tool to check suspicious messages without deep cybersecurity knowledge.
	Type / source of data:	Email and message text samples labelled as legitimate or phishing (INSERT ACTUAL DATASET NAME / SOURCE HERE)



Key Focus

Question:

What problem were we trying to solve, and why did we think this was a suitable problem for experimenting with AI?

Answer:

Phishing emails feature highly varied wording, tone, and evolving context, making static rules unreliable. It is a useful AI experiment because fine-tuning and RAGI let us test how LLMs handle nuanced fraudulent patterns while supporting explainable classifications.

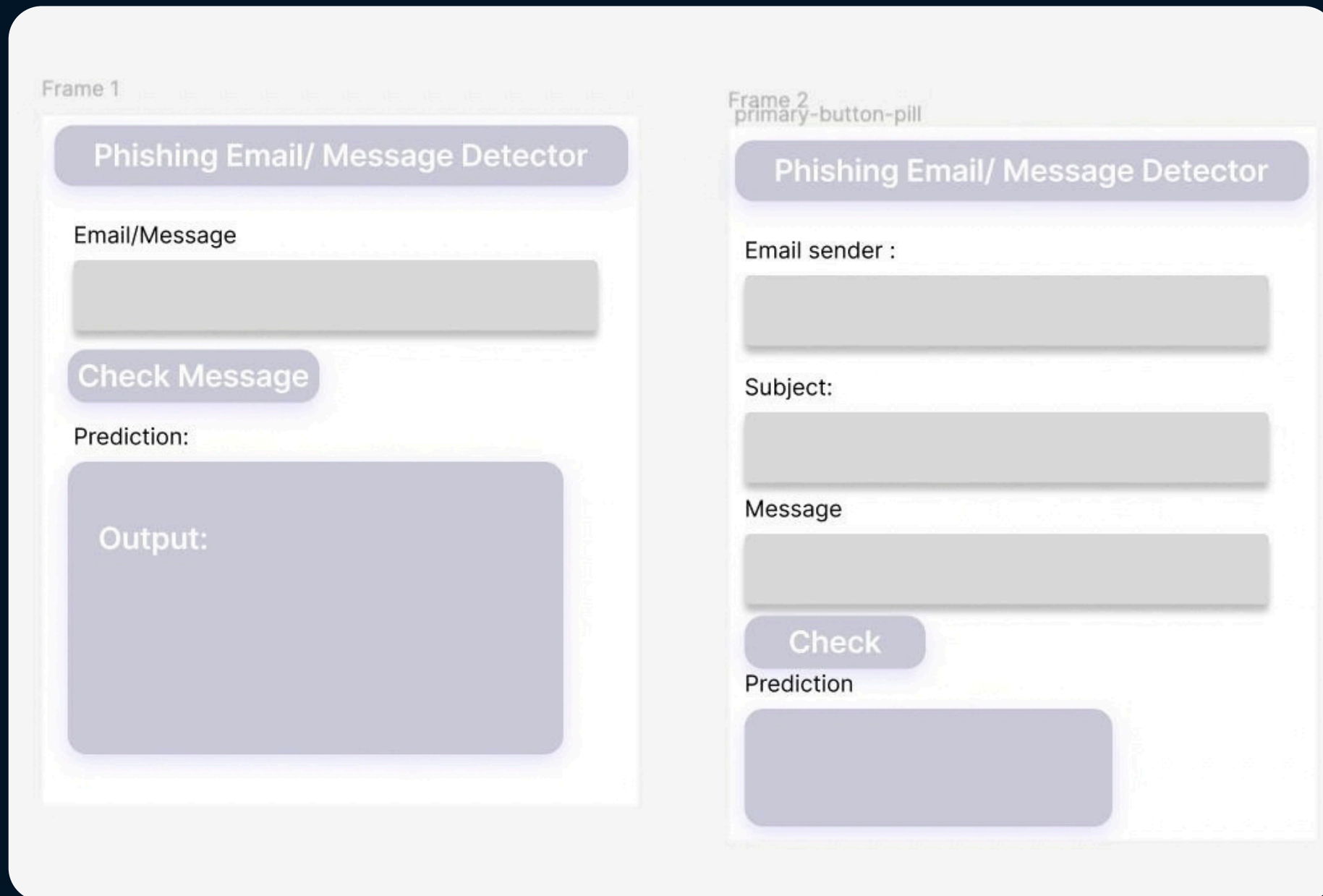


Phase 1: Initial Attempt

What we tried first

We did not build a working Streamlit prototype. Instead, Phase 1 consisted of:

1. Exploring and preparing the dataset - ([kaggle.dataset](#)) cleaning, labeling, and formatting emails/messages into phishing vs. legitimate examples.
2. Designing the intended user interface - an Figma mockup of what the detector would look like (two variants: a single-message detector and a sender/subject/message detector).
3. Going straight into fine-tuning Qwen 1B on the prepared data



DEMO

Phase 2

Preparing data, training approaches, and collecting evidence



DATA



Dataset source

- Primary source: Enron.csv from the Kaggle Phishing Email Dataset ([phishing-email-dataset](#))
- We also explored simlab-vs/meajor_cleaned_preprocessed on Hugging Face
- We ended up using only the Enron.csv dataset for fine-tuning because it had a clean, simple structure (subject, body, label) and was already labeled for phishing vs. legitimate.



Records used

29,767 emails – 15,791 legitimate (53%), 13,976 phishing (47%).
Split: 23,813 train / 2,977 test (10%) 2,977 validation (10%).



Cleaning And Preparing steps actually performed

- Filled 198 missing subject lines with empty strings
- Confirmed no duplicates and no completely empty emails
- Dropped temporary length columns (subject_length, body_length) used only for exploration
- Converted each row into an instruction-tuning format:



TRAINING (QLoRA)



Model

Qwen2.5 1.5B Instruct (4-bit quantized)



Purpose

Learn our exact output format (single label, not a paragraph) and Enron-specific phishing patterns.



Training example used

```
def converted_example(example):  
    if example["label"] == 1:  
        label = "legitimate"  
    else:  
        label = "phishing"  
  
    return {  
        "instruction": "Classify the following email as phishing or legitimate.",  
        "input": f"Subject: {example['subject']}\n\nBody: {example['body']}",  
        "output": label  
    }
```



Training setup / compute limitation

Google Colab free-tier GPU. Limited session time and VRAM restricted batch size and training duration(Tesla T4 (14.56 GB VRAM))



EVIDENCE: BEFORE VS AFTER

Base Qwen – output distribution

```
print("Unique predictions:")  
print(set(predictions))  
  
print("\nUnique references:")  
print(set(references))  
  
Unique predictions:  
{'x x x x x', 'the following :\r\ndate :'  
  
Unique references:  
{'legitimate', 'phishing'}
```

Fine-tuned – output distribution

```
Both 'max_new_tokens' (=5) and 'max_length' (=32768) seem to h  
EMAIL:  
Subject: = ? utf - 8 ? q ? are you pimped with ? =  
  
Body: = ? utf - 8 ? q ? a rolex watch ? ? =  
veritable replications wristwatches  
we offer next trademarks available also :  
bvlgari  
chopard  
iwc  
zenith  
omega  
  
EXPECTED LABEL:  
legitimate  
  
FINE-TUNED MODEL PREDICTION:  
legitimate
```



What we observed

Same test set (2977 samples). Before fine-tuning, outputs were unpredictable 300+ distinct strings, none valid labels. After fine-tuning, outputs collapsed to {legitimate, phishing} with 99.2% accuracy on valid predictions.



Note

242 invalid outputs (8.1%) – the model still sometimes produces non-labels. Counting invalid as wrong, end-to-end accuracy is ~91.2%



Key takeaway

We prepared our data, trained approaches under documented settings, and collected before/after evidence on the same example.
Next: how the RAG system works and how we diagnose failures.

 Qwen for Phishing Detection – Evidence-Led Study

Slide 3 of 9

Phase 2:

Limitations

```
print(f"Accuracy:  {accuracy * 100:.2f}%")
print(f"Precision: {precision * 100:.2f}%")
print(f"Recall:    {recall * 100:.2f}%")
print(f"F1 Score:  {f1 * 100:.2f}%")
```

Accuracy: 99.20%
Precision: 99.65%
Recall: 98.82%
F1 Score: 99.23%

Limitation	Observed
Training data quality	198 missing subjects; some noisy labels
Compute	Colab T4 — 0.35 epochs, batch 1
Hallucinations	242 invalid outputs (8.1%)
Overfitting	Train 0.28 vs val 1.69

What did we learn that we couldn't learn from the prototype?
Fine-tuning changes behavior, not knowledge. It taught the model our format and dataset patterns, but couldn't give it new information. That's the gap RAG fills in Phase 3.

Phase 3 | Retrieval-Augmented Generation (RAG) + Fine-Tuned Qwen Pipeline System

Retrieval-Augmented Generation for phishing email classification

RAG PIPELINE OVERVIEW



✓ Key idea: RAG retrieves similar emails as supporting evidence before Qwen makes the final classification.

Limitations

- Similar emails $\neq$ same classification
- Retrieved evidence can sometimes be irrelevant
- Qwen may not always use retrieved evidence correctly
- Generative model can produce invalid outputs

Model Evaluation

Performance comparison of the three phishing detection approaches.

Model Comparison on 2,977 Emails

All three models were evaluated on the same 2,977-email test set.

Model	Accuracy	Precision	Recall	F1-Score	Invalid Predictions
TF-IDF + Logistic Regression	98.62%	97.50%	99.65%	98.56%	0
Fine-Tuned Qwen	99.20%	99.65%	98.82%	99.23%	242
RAG + Qwen	98.02%	97.15%	98.70%	97.92%	55

EVIDENCE FLOW

USER EMAIL

Analyze an Email

Enter the email content:

Hello,

We were unable to process the payment method associated with your account during our latest billing cycle.

Please review your payment information to make sure your account remains active. You can update your details through the account portal at your earliest convenience.

If you have already updated your payment information, no further action is required.

Best regards,

 Analyze Email

RETRIEVED INFORMATION

RAG Retrieved Evidence

These emails were retrieved from the ChromaDB knowledge base and provided to Qwen as supporting evidence.

> Evidence 1 — phishing

> Evidence 2 — phishing

> Evidence 3 — phishing

RAG Retrieved Evidence

These emails were retrieved from the ChromaDB knowledge base and provided to Qwen as supporting evidence.

▼ Evidence 1 — phishing

Retrieved label: phishing

Distance: 0.7236

Retrieved email 1

Subject: urgent visa update

Body: dear visa cardholder .
it has come to our attention that your visa billing information records are out of date . this requires an update of your billing information . please take several minutes out of your online experience and update your billing records . you will not run into future problems with our services . please update your

FINAL ANSWER

 **RAG + Fine-Tuned Qwen**

 **PHISHING**



Evaluation

How we judge success



CONFUSION OUTCOMES



True positive

Phishing correctly detected as phishing



False positive

Legitimate incorrectly flagged as phishing



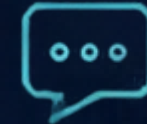
True negative

Legitimate incorrectly flagged as phishing



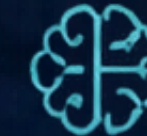
False negative

Phishing missed and marked as legitimate



What test data did we use?

We used a held-out test set of 2,977 emails (split 50/50 from the original test.jsonl with seed=42), containing a balanced mix of 1,411 legitimate and 1,566 phishing emails.



What made an answer "good"?

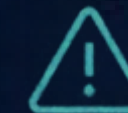
An answer was considered "good" if it was a valid label (phishing or legitimate) that matched the ground-truth label. We measured performance using Accuracy, Precision, Recall, and F1-Score.



Same inputs

Yes. We evaluated three different approaches on the exact same 2,977 emails to ensure a fair comparison:

- TF-IDF + Logistic Regression (Classic ML baseline)
- Fine-Tuned Qwen (Fine-tuned LLM)
- RAG + Qwen (Retrieval-Augmented Generation)



Limitations?

- The LLMs produced invalid outputs (242 for Fine-tuned Qwen, 55 for RAG+Qwen). These were excluded, which may slightly overstate real-world accuracy. TF-IDF had 0 invalid predictions.
- The test set is from the Enron corpus (early 2000s) and may not reflect modern phishing tactics.
- RAG + Qwen was substantially slower and more computationally expensive than the TF-IDF baseline.



Data privacy, governance, and responsible AI

Our practices for ethical and responsible use



01

MESSAGE PRIVACY

- Emails stored as subject + body text only, in JSONL. Sender/receiver metadata not retained. User-submitted messages are classified in-session and discarded.
- Contains real names and email addresses from the 2001–2002 Enron corpus. Not anonymized in Enron.csv. Could allow identification of individuals.
- Data stored on Google Drive (private group folder). Access limited to 4 group members. Colab had session read-access during training only.



02

DATA PERMISSIONS

- Dataset used were Enron.csv from Kaggle ([phishing_email-dataset](#)). Also explored simlab-vs/meajor_cleaned_preprocessed on HuggingFace.
- HuggingFace dataset: CC BY 4.0. Kaggle Enron.csv: license is [CC BY-SA 4.0](#)
- Attribution provided for both. Share-alike terms may apply to our fine-tuned model as a derivative.



Fine-tuning and RAG have different governance needs based on where information comes from.



03

DEPLOYMENT RISKS

- EXTERNAL SERVICES USED: Google Colab (training + inference), Google Drive (storage), HuggingFace (model + dataset downloads), Unsloth (local software dependency). No data uploaded to HuggingFace.
- ACTUAL PRACTICE FOR THIRD-PARTY RISK: Data stays on Drive; Colab sessions are ephemeral. ChromaDB runs locally on Drive.
- POTENTIAL RISKS IDENTIFIED:
 - Colab sessions give Google temporary access to data.
 - Fine-tuning may allow extraction of real names/emails from training data
 - Unsloth is a supply-chain dependency, trusted third-party code patching our training pipeline.



Governance differs for fine-tuning (model learns from training data) and RAG (uses external knowledge records). Our practices reflect this distinction and will be updated as the project evolves.



What the development process taught us



01



Prototype

- [WHAT WE EXPECTED]: To quickly connect a pre-trained Qwen model to a simple UI and get accurate phishing detections.
- [WHAT WE OBSERVED]: The base model was inconsistent. It often failed to follow the output format, hallucinating random text (e.g., 'x x x x x') instead of 'phishing' or 'legitimate'.
- [WHAT WE CHANGED]: Realized a general-purpose model wasn't enough. We needed to specialize it. This forced us to shift focus to **Phase 2: Fine-tuning**.

02



Qwen fine-tuning

- [WHAT WE EXPECTED]: Retrieving similar emails from ChromaDB would act as a cheat sheet, pushing accuracy to near 100%.
- [WHAT WE OBSERVED]: Retrieval worked well (Recall@1: 98.4%). After improving the RAG prompt to use Qwen's chat template properly, RAG+Qwen reached 98.02% accuracy, significantly better than our earlier attempt (90.5%), but still below fine-tuned Qwen (99.05%). Interestingly, RAG produced fewer invalid outputs (55) than fine-tuning alone (241).
- [WHAT WE CHANGED]: RAG improves output stability but does not beat a well-fine-tuned model for classification accuracy.

03



RAG

- [WHAT WE EXPECTED]: Retrieving similar emails from ChromaDB would act as a cheat sheet, pushing accuracy to near 100% and eliminating hallucinations.
- [WHAT WE OBSERVED]: Retrieval worked well (Recall@1: 98.4%), but RAG+Qwen accuracy dropped to 90.5% — much lower than fine-tuned Qwen (99.2%). Misleading retrieved examples confused the model.
- [WHAT WE CHANGED]: Learned that RAG is not a magic fix. The quality of the retriever directly dictates the quality of the generator. (Next step would be re-ranking retrieved results).



What the development process taught us



MOST USEFUL FEEDBACK

The limitations of the prototype (random outputs) were the most useful feedback. They proved that fine-tuning was absolutely necessary and saved us from wasting time building a complex UI on top of a broken model.



EARLY PROBLEM THAT AFFECTED LATER WORK

The base model's tendency to hallucinate random strings during the prototype phase was never fully solved. Even after fine-tuning, 241 invalid outputs remained (8.1%). RAG+Qwen reduced this to 55 invalid outputs, but RAG still could not surpass fine-tuned Qwen's accuracy (98.0% vs. 99.05%)



WHAT WE WOULD TEST EARLIER

We should have tested the retrieval quality before fine-tuning Qwen. If we had known retrieval worked so well, we might have fine-tuned the model specifically to interpret the retrieved evidence, rather than just classify raw text.



Final reflection

01



We learned...

FINE-TUNING IS FOR CHANGING BEHAVIOR (TEACHING THE MODEL HOW TO ANSWER), WHILE RAG IS FOR PROVIDING KNOWLEDGE (GIVING THE MODEL WHAT TO LOOK AT). WE NEEDED BOTH

02



We were surprised by...

THE BASE MODEL HALLUCINATED RANDOM STRINGS. EVEN AFTER FINE-TUNING, 241 INVALID OUTPUTS (8.1%) REMAINED. SURPRISINGLY, ADDING RAG ACTUALLY IMPROVED PERFORMANCE COMPARED TO THE EARLIER ATTEMPT — RAG+QWEN REACHED 98.0% ACCURACY ON VALID PREDICTIONS, THOUGH STILL BELOW FINE-TUNED QWEN'S 99.05%.

03



Next, we would...

REVERSE THE ORDER: BUILD AND VALIDATE THE RAG RETRIEVER FIRST, THEN FINE-TUNE THE MODEL TO DIGEST THE RETRIEVED EVIDENCE.



Our project compared different ways to support phishing detection; the most suitable approach depends on measured errors, data quality, and the needs of the user.



Thank you

Questions?



<https://github.com/Swostika-Lama/PhishDetectAI.git>

